EXPERIMENT 23

Title: Write a Prolog Program to Implement Family Tree

Aim

To develop a Prolog program to represent a family tree and determine relationships between family members using facts, rules, and logical queries.

Objective

- To understand the representation of relationships in Prolog.
- To define family relationships using facts and rules.
- To implement predicates such as parent, sibling, grandparent, uncle, and aunt.
- To retrieve family relationships using Prolog queries.
- To understand logical inference in Artificial Intelligence.

Theory

A family tree can be represented in Prolog using facts and rules. Facts are used to represent basic relationships such as parent, while rules are used to derive other relationships such as grandparent, sibling, uncle, and aunt.

For example:

parent(ravi, arun). means that Ravi is

the parent of Arun.

Using this fact, Prolog can derive other relationships through rules. This demonstrates how a knowledge base can be used for logical reasoning.

Algorithm

Step 1: Start the program.

Step 2: Define facts representing parents and genders.

Step 3: Define rules for different family relationships.

Step 4: Define rules for sibling, grandparent, uncle, and aunt relationships.

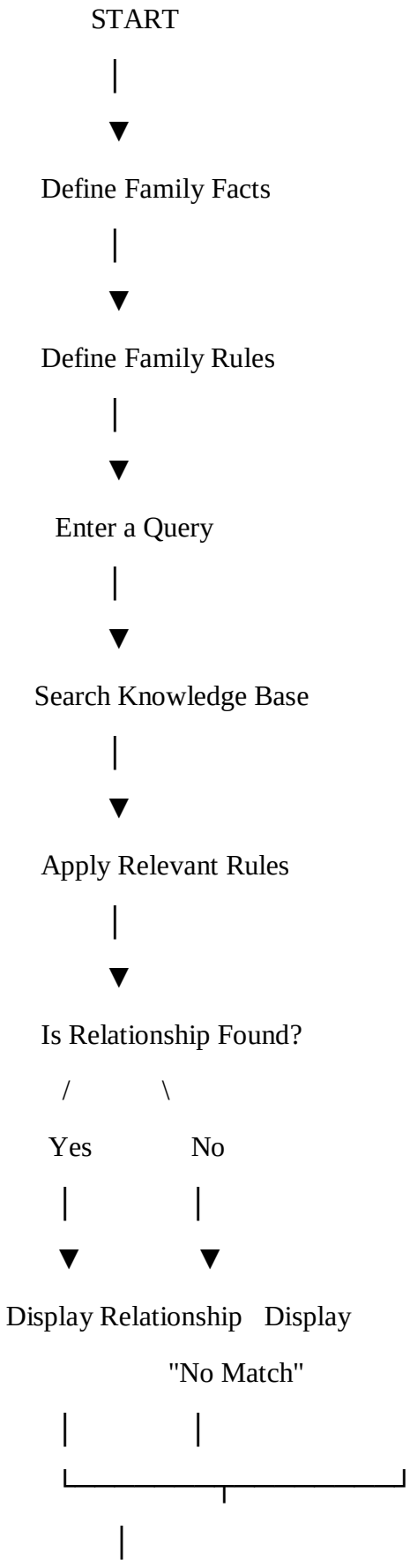
Step 5: Enter a query for the required relationship.

Step 6: Prolog searches the knowledge base and applies the rules.

Step 7: Display the resulting family relationship.

Step 8: Stop the program.

Flowchart


START
Define Family Facts
Define Family Rules
Enter a Query
Search Knowledge Base
Apply Relevant Rules
Is Relationship Found?
Yes
No
Display Relationship
Display "No Match"

▼

STOP

Prolog Program

```
% Family Tree Database

% Parent relationships
parent(ravi, arun). parent(ravi,
priya). parent(meena, arun).
parent(meena, priya).

parent(arun, rahul). parent(arun,
kavya).

parent(priya, rohan). parent(priya,
divya).

% Gender facts
male(ravi). male(arun).
male(rahul).
male(rohan).

female(meena).
female(priya). female(kavya).
female(divya).

% Rules
```

```
% Child relationship child(Child,
Parent) :-     parent(Parent,
Child).

% Sibling relationship
sibling(X,      Y)      :-
parent(P,               X),
parent(P, Y),
   X \= Y.

% Grandparent relationship
grandparent(GP, GC) :-
parent(GP, P),     parent(P,
GC).

% Grandchild relationship
grandchild(GC, GP) :-
grandparent(GP, GC).

% Brother relationship
brother(X,      Y)      :-
male(X),      sibling(X,
Y).

% Sister relationship
sister(X,      Y)      :-
female(X),
sibling(X, Y).
```

```
% Uncle relationship
uncle(U, N) :-
male(U),     sibling(U,
P),     parent(P, N).

% Aunt relationship
aunt(A, N) :-
female(A),     sibling(A,
P),     parent(P, N).
```

Queries

Query 1: Find the children of Ravi

```
?- parent(ravi, Child).
```

Sample Output

```
Child = arun ;

Child = priya ; false.
```

Query 2: Find the siblings of Arun

```
?- sibling(arun, Sibling).
```

Sample Output

```
Sibling = priya.
```

Query 3: Find the grandchildren of Ravi

```
?- grandparent(ravi, Grandchild).
```

Sample Output

```
Grandchild = rahul ;

Grandchild = kavya ;

Grandchild = rohan ;
```

Grandchild = divya ; false.

Query 4: Find the parent of Rahul

?- child(rahul, Parent).

Sample Output

Parent = arun.

Query 5: Find the sisters of Arun

?- sister(Sister, arun).

Sample Output

Sister = priya.

Query 6: Find the grandchildren of Meena

?- grandparent(meena, Grandchild).

Sample Output

Grandchild = rahul ;

Grandchild = kavya ;

Grandchild = rohan ;

Grandchild = divya ; false.

Explanation

The parent/2 predicate represents the basic parent-child relationship.

For example: parent(ravi, arun).

means Ravi is the parent of

Arun.

The sibling/2 rule determines whether two people have the same parent.

The grandparent/2 rule determines whether one person is the parent of another person's parent.

The brother/2 and sister/2 rules use gender information along with the sibling relationship.

Similarly, the uncle/2 and aunt/2 rules identify the siblings of a person's parent.

Thus, Prolog can derive relationships that are not directly stored as facts.

Advantages

- Easy representation of family relationships.
- Demonstrates logical reasoning using rules.
- Easy to add new family members.
- Supports multiple relationship queries.
- Useful for understanding knowledge representation.

Applications

- Family relationship systems.
- Knowledge representation.
- Expert systems.
- Artificial Intelligence applications.
- Rule-based reasoning systems.
- Relationship-based databases.

Result

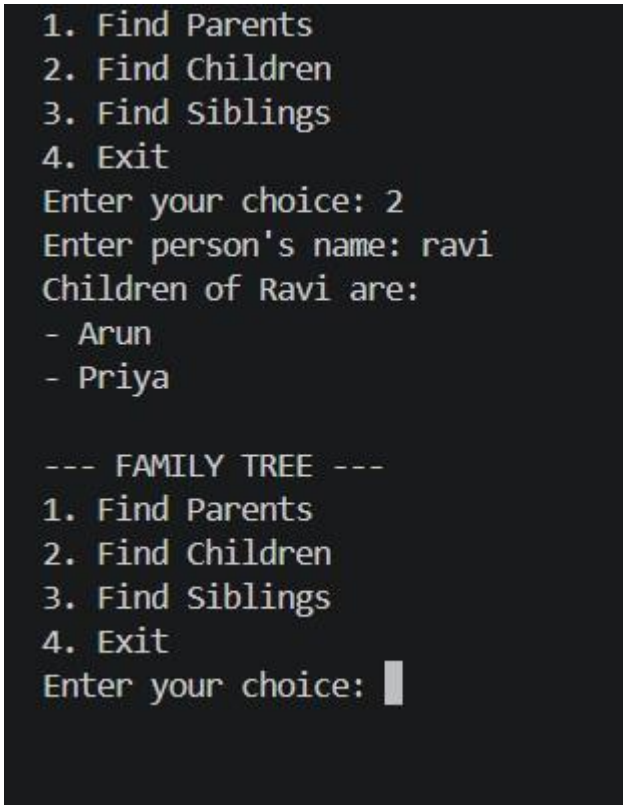
The Prolog program was successfully implemented to represent a family tree and determine various relationships such as parent, child, sibling, grandparent, brother, and sister using appropriate queries.

Conclusion

Thus, the family tree was successfully implemented using Prolog. This experiment demonstrates how facts and logical rules can be combined to represent relationships and derive new information from an Artificial Intelligence knowledge base.

```
def find_siblings(person):
            for child in children:
                if child != person:
                    siblings.add(child)

    if siblings:
        print("Siblings of", person, "are:")
        for sibling in siblings:
            print("-", sibling)
    else:
        print("No siblings found.")


# Main program
while True:
    print("\n--- FAMILY TREE ---")
    print("1. Find Parents")
    print("2. Find Children")
    print("3. Find Siblings")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        person = input("Enter person's name: ").capitalize()
        find_parents(person)

    elif choice == "2":
        person = input("Enter person's name: ").capitalize()
        find_children(person)

    elif choice == "3":
        person = input("Enter person's name: ").capitalize()
        find_siblings(person)

    elif choice == "4":
        print("Program terminated.")
        break

    else:
        print("Invalid choice.")
```

```python
# FAMILY TREE

# Parent relationships
parents = {
    "Ravi": ["Arun", "Priya"],
    "Meena": ["Arun", "Priya"],
    "Arun": ["Karthik", "Divya"],
    "Priya": ["Karthik", "Divya"]
}


# Find parents of a person
def find_parents(person):
    found = False

    for parent, children in parents.items():
        if person in children:
            print(parent, "is a parent of", person)
            found = True

    if not found:
        print("Parent information not found.")


# Find children of a person
def find_children(person):
    if person in parents:
        print("Children of", person, "are:")
        for child in parents[person]:
            print("-", child)
    else:
        print("No children found.")


# Find siblings of a person
def find_siblings(person):
    siblings = set()

    for parent, children in parents.items():
        if person in children:
            for child in children:
                if child != person:
```


1. Find Parents
2. Find Children
3. Find Siblings
4. Exit
Enter your choice: 2
Enter person's name: ravi
Children of Ravi are:
- Arun
- Priya
--- FAMILY TREE ---
1. Find Parents
2. Find Children
3. Find Siblings
4. Exit
Enter your choice: